TrustRAG: Cheap Claim-Level Hallucination Detection for Retrieval-Augmented Generation

Can a 0.4B encoder catch what a 70B judge catches, at a fraction of the cost?

Adhithyan V (Roll No. 230121003)
Aditya Raj (Roll No. 230121004)

---

1. Motivation

Large language models make things up. The field calls this hallucination, and it has been studied at length [1]. Retrieval-Augmented Generation, or RAG [2], was meant to fix it. The idea is simple. Give the model the source documents. Ask it to answer from them. It should then stop inventing facts, but it doesn't.

RAGTruth [3] is a dataset of about 18,000 RAG answers. Human annotators marked every unsupported word. They found unsupported text in a large share of answers, even when the correct evidence had been supplied. GPT-4 did this. So did Llama-2 and Mistral. A recent survey of RAG names this the main open problem in the area [14].

Two things go wrong. Sometimes the model contradicts the evidence. More often it adds something extra: a date, a name, a reason. The addition sounds correct and fits the topic. It is simply not in the source.

This matters because of where RAG gets used. Hospitals. Law firms. Banks. Government helplines. In those places a made-up sentence is worse than no answer, because a confident paragraph stops people from checking. "I could not find this in the documents" makes them check.

Today's tools have three problems.

- **They are too coarse.** Most systems give one score for the whole answer. If sentence two is wrong, you still get one number. You cannot tell which part to distrust.
- **They are too expensive.** LLM-as-judge calls a large model for every answer. SelfCheckGPT [5] needs several samples. FActScore [6] runs one query per fact. All of these cost seconds and money.
- **They do not act.** The score goes to a dashboard. A human reads it days later. The bad answer was still shown to the user.

We want to fix all three at once. Our approach builds on earlier work showing that natural language inference (NLI) models are cheap and strong at checking factual consistency [7, 8], and on selective prediction [9], which turns a confidence score into a rule for when to stay silent.

2. Problem Description

A user asks a question q. The system retrieves evidence E, a set of passages. A model writes an answer a. We split a into claims c1 to cm. A claim is unsupported if E does not back it. We label it y = 1 in that case, and y = 0 otherwise. We want to learn a function f(c, E) that predicts this label.

The project has four parts.

1. **Detection.** Predict which claims are unsupported. Measured by AUROC, AUPRC and F1.

2. **Calibration.** Make the score a real probability. A score of 0.8 should mean the claim is wrong about 80% of the time. Measured by Expected Calibration Error and Brier score.
3. **Selective prediction.** Use the score to decide: show the claim, flag it, or hold it back. Measured by the risk-coverage curve.
4. **Cost.** Do all of the above cheaply. Measured in milliseconds per answer, GPU memory, and cost per 1,000 queries.

2.1 Our main idea

NLI models are trained on one question: does sentence B follow from sentence A? That is exactly our question. So a small NLI model may do this job about as well as a very large language model. It does not need to reason or write. It only needs to compare two short sentences. If we are right, checking answers becomes cheap enough to run on every response instead of a sample. If we are wrong, we will still learn which kinds of hallucination need a big model to catch. Either outcome tells us something worth knowing.

2.2 Limits and objectives

Everything must run on one free Colab T4 with 16 GB of memory. Every part of the system must be open-weight. A 70B model is used only as a comparison, never as a component. We do not try to fix cases where the answer was never in the corpus, and we do not cover other languages, images, or fine-tuning the generator itself.

Our objectives are:

1. Build the claim splitter and the NLI checker, and get a first detection score on RAGTruth.
2. Break the results down by each of the four RAGTruth hallucination types.
3. Combine six cheap signals into one calibrated probability.
4. Compare against six baselines, from simple word overlap up to a 70B LLM judge.
5. Train on RAGTruth and test on HaluEval without changes, to show the method transfers.
6. Build a small demo that colours each claim green, amber or red and shows its evidence.

3. Dataset to be Used

The choice of data is what makes this project practical. Both main datasets already contain LLM answers that have been generated and labelled by humans, together with the exact evidence the model was given. So we do not have to run a generator to get our main result. We only score answers that already exist, which removes the slowest and least reliable step from the pipeline.

Dataset	Size and access	Labels	Why we use it
RAGTruth [3]	15,090 train, 2,700 test. HF: wandb/RAGTruth-processed	Character-level spans marking unsupported text, with four type labels. Answers come from GPT-4, GPT-3.5, Llama-2 and Mistral.	Main dataset. Trains and tests the detector. The type labels let us say which kinds of error we catch and which we miss.
HaluEval [4]	10,000 QA rows. HF: pminervini/HaluEval, config 'qa'	Every question has one correct answer and one hallucinated answer, plus the supporting text. The classes are balanced.	Transfer test. We never train on it. It shows whether the method works on a different generator and format.
HotpotQA [10]	113k multi-hop questions. Each has 2 correct and 8 distractor paragraphs.	Answer spans and sentence-level supporting facts.	Live demo. We index 500 documents and run our own retriever and generator, so the demo checks fresh text.
FEVER [11]	185k claims over Wikipedia.	SUPPORTED / REFUTED / NOT ENOUGH INFO, with evidence sentences.	Used indirectly. Our NLI model was already trained on it, which is why it works on our task without extra training.

Table 1. The four datasets and the job each one does.

Preprocessing is short. We split answers and evidence into sentences with spaCy. A claim is labelled unsupported if at least half of its characters fall inside a marked span. We check borderline cases by hand. We split train and validation by source document, not by response. This detail matters: splitting by response puts the same evidence passage on both sides of the split, which would leak information and make our numbers look better than they are.

4. Methodology

4.1 The pipeline

The system runs in five steps.

1. **Split.** Cut the answer into sentence-level claims. Cut the evidence into sentences too.
2. **Check.** For every claim and every evidence sentence, run an NLI model. We use DeBERTa-v3-large-mnli-fever-anli-ling-wanli, which has 0.4B parameters [12]. It returns three probabilities: entailment, neutral and contradiction.
3. **Score.** For each claim, take the best evidence sentence. The support score is the highest value of P(entailment) minus P(contradiction) across all evidence sentences.
4. **Combine.** Feed six features into a logistic regression, then calibrate the output with isotonic regression [13]. This gives a probability that the claim is unsupported.
5. **Decide.** Two thresholds split that probability into three actions. A low score means show the claim. A middle score means show it but flag it and attach the evidence. A high score means hold the claim back and mark the answer as incomplete.

One design choice is worth explaining. We compare each claim against single evidence sentences rather than the whole passage. NLI models get much weaker when the input text is long. Scoring short sentences and keeping the best result avoids most of that loss. We include the whole-passage version as a baseline so we can measure how much this choice is worth.

4.2 The six features

Every feature is cheap. All of them come from one NLI pass plus one embedding pass.

1. Support score: the best value of P(entailment) minus P(contradiction).
2. Peak contradiction: the highest P(contradiction). This separates a direct conflict from a claim that is simply missing from the evidence.
3. Mean entailment over the top three evidence sentences.
4. Highest cosine similarity between the claim and any evidence sentence, using bge-small.
5. Claim specificity: how many names and numbers the claim contains. Specific claims are made up more often than vague ones.
6. Claim length in tokens, to keep the other features fair across long and short claims.

We use logistic regression rather than a neural layer on purpose. With six features we can read the weights directly. That lets us say which signal is doing the work, instead of quoting a number we cannot explain.

4.3 Baselines and metrics

Baseline	What it tells us
Random guess	The floor. HaluEval is balanced, so this scores 0.50.
Word overlap (ROUGE-L)	How much of the task is solved by matching words.
Embedding similarity	Whether similarity alone is enough, without entailment.
NLI on the whole passage	How much sentence-level scoring actually helps.
Llama-3.2-3B as judge	Whether a small generative model beats a small encoder.
Llama-3.3-70B as judge	The ceiling, and the cost we are trying to avoid.

Table 2. Six baselines, from trivial to expensive. The 70B judge runs on a fixed sample of 500 examples. Every other method is scored on that same sample so the comparison is fair.

Here is what we measure.

- **Detection:** AUROC, AUPRC and F1, overall and split by hallucination type.
- **Calibration:** Expected Calibration Error with 15 bins, plus Brier score.
- **Selective prediction:** risk-coverage curve, and the error rate left at 90% coverage.
- **Transfer:** train on RAGTruth, test on HaluEval, and see how much accuracy we lose.
- **Cost:** milliseconds per answer, peak GPU memory, and cost per 1,000 queries.
- **Human check:** both of us label the same 100 claims, then compare our labels with Cohen's kappa.

We also plan ablations: verifier size (0.4B against 0.18B), pooling method, dropping each feature in turn, and results split by which model wrote the answer. We compare against RAGAS [15], the standard RAG metric, and discuss Self-RAG [16] as an alternative that checks during generation rather than after.

4.4 Risks

- **Bad sentence splitting.** If the splitter cuts claims badly, our detector will look worse than it is. We will check 100 splits by hand and measure an upper bound using correct splits.
- **Colab timeouts.** Long runs can die. Every scoring script saves progress to Drive every 200 examples and resumes from there.
- **Class imbalance.** Unsupported claims are the minority in RAGTruth. We track AUPRC alongside AUROC, use class weights, and use the balanced HaluEval set as a check.
- **Our idea may be wrong.** If NLI loses badly to the 70B judge, we say so. The per-type results will show which errors need a large model, which is a useful finding on its own.

5. Expected Outcomes and Relevance

We expect a detection AUROC of roughly 0.78 to 0.88 on RAGTruth, within a few points of the 70B judge but far cheaper per query. We expect calibration error under 0.05, and some drop on the HaluEval transfer test. We also expect one clear weakness. Subtle additions that fit the topic will probably look neutral rather than contradictory, so those should be hardest to catch. Finding that out is part of the point.

5.1 How this helps us now

- **It puts the whole LLM stack into one working system.** Retrieval, transformer encoders, quantisation, calibration and abstention stop being separate topics we read about. They become one thing we had to debug.
- **It teaches evaluation, not just model building.** The hard part is not getting a model to run. It is making splits that do not leak, picking baselines that are hard to beat, and being honest about the number that could prove us wrong.

- **It gives us something concrete to show.** A benchmark table and a working demo are more convincing than a notebook with one accuracy figure.
- **It cannot quietly fail.** The idea is testable either way, so there is no version of this where we end up with nothing to say.

5.2 How this helps us later

- **The work matches real jobs.** LLM evaluation, trust and safety, and AI reliability are named roles at many companies now. Checking RAG answers is what those roles do.
- **It applies directly to deployed systems.** Almost any company that ships a RAG assistant ends up needing this check. Most build one poorly.
- **It can grow into research.** The problem is open and the setup is standard. It extends into other languages, multi-hop claims, or learned claim splitting.
- **It does not go out of date.** The checker wraps any generator. Work that fine-tunes one specific model does not survive being replaced.

6. References

[1] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., and Fung, P. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, 55(12), 1-38. https://arxiv.org/abs/2202.03629

[2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459-9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[3] Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., and Zhang, T. (2024). RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 10862-10878. https://aclanthology.org/2024.acl-long.585/

[4] Li, J., Cheng, X., Zhao, W. X., Nie, J., and Wen, J. (2023). HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. Proceedings of EMNLP 2023, 6449-6464. https://aclanthology.org/2023.emnlp-main.397/

[5] Manakul, P., Liusie, A., and Gales, M. J. F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. Proceedings of EMNLP 2023, 9004-9017. https://aclanthology.org/2023.emnlp-main.557/

[6] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P. W., Iyyer, M., Zettlemoyer, L., and Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. Proceedings of EMNLP 2023, 12076-12100. https://aclanthology.org/2023.emnlp-main.741/

[7] Laban, P., Schnabel, T., Bennett, P. N., and Hearst, M. A. (2022). SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization. Transactions of the Association for Computational Linguistics (TACL), 10, 163-177. https://aclanthology.org/2022.tacl-1.10/

[8] Zha, Y., Yang, Y., Li, R., and Hu, Z. (2023). AlignScore: Evaluating Factual Consistency with a Unified Alignment Function. Proceedings of ACL 2023, 11328-11348. https://aclanthology.org/2023.acl-long.634/

[9] Geifman, Y., and El-Yaniv, R. (2017). Selective Classification for Deep Neural Networks. Advances in Neural Information Processing Systems (NeurIPS), 30, 4878-4887. https://proceedings.neurips.cc/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

[10] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. Proceedings of EMNLP 2018, 2369-2380. https://aclanthology.org/D18-1259/

[11] Thorne, J., Vlachos, A., Christodoulopoulos, C., and Mittal, A. (2018). FEVER: a Large-scale Dataset for Fact Extraction and VERification. Proceedings of NAACL-HLT 2018, 809-819. https://aclanthology.org/N18-1074/

[12] He, P., Gao, J., and Chen, W. (2023). DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2111.09543

[13] Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. Proceedings of the 34th International Conference on Machine Learning (ICML), 1321-1330. https://proceedings.mlr.press/v70/guo17a.html

[14] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997. https://arxiv.org/abs/2312.10997

[15] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. Proceedings of EACL 2024: System Demonstrations, 150-158. https://aclanthology.org/2024.eacl-demo.16/

[16] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. International Conference on Learning Representations (ICLR). https://arxiv.org/abs/2310.11511